Business Case Study - Target

Q1 (1) Data type of all columns in the "customers" table.

Query

```
SELECT column_name, data_type
from Business_Case_Target_SQL.INFORMATION_SCHEMA.COLUMNS
where Table_name = 'customers';
```

Result

Row	column_name	data_type
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

Insights:

There are 5 fields in the customers table with 4 of them having data type as string and customer_zip_code_prefix is having INT64 as its data type.

Recommendations:

N/A

Q1 (2) Get the time range between which the orders were placed.

Query

```
SELECT min(order_purchase_timestamp) as min,
max(order_purchase_timestamp) as max
from Business_Case_Target_SQL.orders;
```

Result

Row	min	max
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Insights:

We can see that time range between which orders were placed is 2016-09-04 21:15:19 UTC and 2018-10-17 17:30:18 UTC

Recommendations:

N/A

Q1 (3) Count the Cities & States of customers who ordered during the given period.

Query

```
SELECT
count(distinct c.customer_city) as city_count, count(distinct c.customer_state) as state_count
from Business_Case_Target_SQL.customers c
inner join Business_Case_Target_SQL.orders o
on c.customer_id=o.customer_id ;
```

Result

Row	city_count	state_count
1	4119	27

Insights:

Customers ordered from 4119 cities and 27 states during the given period

Recommendations:

Customers ordered from almost all states of Brazil but city coverage in terms of orders can be increased. There are 5570 cities in Brazil but customers ordered from 4119 cities. We can identify the potential cities from where orders did not generate and try to give some city-based offers and discounts so that customers from remaining cities should also place the orders.

Q2 (1) Is there a growing trend in the no. of orders placed over the past years?

Query

```
SELECT
extract(year from order_purchase_timestamp) as Year,
count(order_id) as order_count
from Business_Case_Target_SQL.orders
Group by Year
Order by Year;
```

Result

Row	Year	order_count
1	2016	329
2	2017	45101
3	2018	54011

Insights:

Clearly we can see there is a growing trend in number of orders placed over the years. In 2016 number of orders placed were 329 and 45101 & 54011 in 2017 & 2018 respectively.

Recommendations:

Already orders are growing exponentially so would just recommend to continue the good work. A small recommendation would be focus on repeated orders by customer retention & product quality.

Q2 (2) Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Query

```
SELECT
extract(year from order_purchase_timestamp) as Year,
extract(month from order_purchase_timestamp) as Month,
count(order_id) as order_count
from Business_Case_Target_SQL.orders
Group by Year,Month
Order by Year,Month;
```

Result

Row	Year	Month	order_count
1	2016	9	4
2	2016	10	324
3	2016	12	1
4	2017	1	800
5	2017	2	1780
6	2017	3	2682
7	2017	4	2404
8	2017	5	3700
9	2017	6	3245
10	2017	7	4026

Insights:

There is a monthly seasonality over July & August as orders are on a higher side in both month in 2017 & 2018.

Recommendations:
The orders placed in September & October months are less. The marketing campaigns should be increased for these months also discounts should be given in these months.

Q2 (3) During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- **0-6 hrs : Dawn**
- **7-12 hrs : Mornings**

- **13-18 hrs : Afternoon**
- **19-23 hrs : Night**

Query

```
Select Time_of_the_day,count(order_id) as Number_of_orders from
(SELECT
CASE
   WHEN EXTRACT(HOUR FROM order_purchase_timestamp) Between 0 AND 6 THEN 'Dawn'
   WHEN EXTRACT(HOUR FROM order_purchase_timestamp) Between 7 AND 12 THEN 'Mornings'
   WHEN EXTRACT(HOUR FROM order_purchase_timestamp) Between 13 AND 18 THEN 'Afternoon'
   WHEN EXTRACT(HOUR FROM order_purchase_timestamp) Between 19 AND 23 THEN 'Night'
   END as Time_of_the_day,
order_id
FROM Business_Case_Target_SQL.orders) t
GROUP BY Time_of_the_day
ORDER BY Number_of_orders DESC;
```

Results

Row	Time_of_the_day	Number_of_orders
1	Afternoon	38135
2	Night	28331
3	Mornings	27733
4	Dawn	5242

Insights:

Most of the orders placed by Brazilian customers is in the Afternoon. Mornings & Night orders seems to be decent.

Recommendations:

Orders placed in Dawn is less as expected but it can be increased by giving some discounts in milk products. Marketing campaigns & ads should be run in Night time to increase Night sales.

Q3 (1) Get the month on month no. of orders placed in each state.

Query

```
WITH Location AS (
SELECT geolocation_zip_code_prefix,
MIN(geolocation_state) AS geolocation_state
FROM Business_Case_Target_SQL.geolocation
GROUP BY geolocation_zip_code_prefix
),
```

```
Join_Tables AS (
SELECT Location.geolocation_state,o.order_id,o.order_purchase_timestamp
FROM Location
LEFT JOIN Business_Case_Target_SQL.customers c
ON Location.geolocation_zip_code_prefix = c.customer_zip_code_prefix
LEFT JOIN Business_Case_Target_SQL.orders o
ON c.customer_id = o.customer_id
WHERE o.order_purchase_timestamp IS NOT NULL
),

Year_Month AS (
SELECT geolocation_state,
EXTRACT(YEAR FROM order_purchase_timestamp) AS year,
EXTRACT(MONTH FROM order_purchase_timestamp) AS month_number,
FORMAT_TIMESTAMP('%B', order_purchase_timestamp) AS month_name,
order_id
FROM Join_Tables
)

SELECT geolocation_state,year,month_name,
COUNT(DISTINCT order_id) AS number_of_orders
FROM Year_Month
GROUP BY geolocation_state, year, month_number, month_name
ORDER BY year, month_number;
```

Result

Row	geolocation_state	year	month_name	number_of_orders
1	SP	2016	September	2
2	RR	2016	September	1
3	RS	2016	September	1
4	SP	2016	October	113
5	RJ	2016	October	56
6	ES	2016	October	4
7	MG	2016	October	40
8	BA	2016	October	4
9	SE	2016	October	3
10	PE	2016	October	7

Insights:

The highest number of orders are generated from SP state. MG & RJ are also providing decent number of orders. AP & RR are giving lease orders.

Recommendations:

Identify reasons for low orders in underperforming states and apply successful strategies from high-performing ones. Reallocate more resources to boost growth in these regions.

Q3 (2) How are the customers distributed across all the states?

Query

```
WITH location AS (
SELECT geolocation_zip_code_prefix,
MIN(geolocation_state) AS geolocation_state
FROM Business_Case_Target_SQL.geolocation
GROUP BY geolocation_zip_code_prefix
),

customer AS (
SELECT DISTINCT c.customer_id,l.geolocation_state
FROM Business_Case_Target_SQL.customers c
JOIN location l ON c.customer_zip_code_prefix = l.geolocation_zip_code_prefix
),

orders AS (
SELECT DISTINCT customer_id
FROM Business_Case_Target_SQL.orders
)

SELECT
c.geolocation_state,COUNT(c.customer_id) AS Num_of_customers
FROM customer c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.geolocation_state
ORDER BY Num_of_customers DESC;
```

Result:

Row	geolocation_state	Num_of_customers
1	SP	41711
2	RJ	12820
3	MG	11624
4	RS	5462
5	PR	5034
6	SC	3637
7	BA	3371
8	ES	2027
9	GO	2006
10	DF	1974

Insights:

The highest number of orders are generated from SP state. MG & RJ are also providing decent number of orders. AP & RR are giving lease orders. There are no repeat orders from any customer.

Recommendations:

Identify reasons for low orders in underperforming states and apply successful strategies from high-performing ones. Reallocate more resources to boost growth in these regions. Focus should be on customer retention as repeat orders are zero.

Q4 (1) Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only). You can use the "payment_value" column in the payments table to get the cost of orders.

Query

```
with orders_2017 as
(SELECT Sum(p.payment_value) as cost_2017
from Business_Case_Target_SQL.payments p join Business_Case_Target_SQL.orders o
on p.order_id = o.order_id
where order_purchase_timestamp BETWEEN '2017-01-01' AND '2017-08-31 23:59:59'),

orders_2018 as
(SELECT Sum(p.payment_value) as cost_2018
from Business_Case_Target_SQL.payments p join Business_Case_Target_SQL.orders o
on p.order_id = o.order_id
where order_purchase_timestamp BETWEEN '2018-01-01' AND '2018-08-31 23:59:59')

SELECT Round(orders_2018.cost_2018,2) as cost_2018,Round(orders_2017.cost_2017,2) as
cost_2017,
ROUND((orders_2018.cost_2018 - orders_2017.cost_2017) * 100 / orders_2017.cost_2017, 2) AS
percentage_change
FROM orders_2017,orders_2018;
```

Result

Row	cost_2018	cost_2017	percentage_change
1	8694733.84	3669022.12	136.98

Insights:

There is a percentage increase of almost 137% from 2017 to 2018 for Jan to Aug Month.

Recommendations:

As there is a good rise in payment value would just recommend to continue the same work with little more efforts to make percentage increase rise to 150% to 160%.

Q4 (2) Calculate the Total & Average value of order price for each state.

Query

```
WITH location AS (
SELECT DISTINCT geolocation_zip_code_prefix, geolocation_state
FROM Business_Case_Target_SQL.geolocation
),

customer AS (
SELECT customer_id, customer_zip_code_prefix
FROM Business_Case_Target_SQL.customers
),

orders AS (
SELECT o.order_id, o.customer_id, p.payment_value
FROM Business_Case_Target_SQL.orders o
JOIN Business_Case_Target_SQL.payments p
ON o.order_id = p.order_id
),

customer_rank AS (
SELECT c.customer_id,l.geolocation_state,
ROW_NUMBER() OVER (PARTITION BY c.customer_id ORDER BY l.geolocation_state) AS rn
FROM customer c JOIN location l
ON c.customer_zip_code_prefix = l.geolocation_zip_code_prefix
),

rank_1_customer AS (
SELECT customer_id, geolocation_state
FROM customer_rank
WHERE rn = 1
)

SELECT
rank_1_customer.geolocation_state,
ROUND(SUM(orders.payment_value), 2) AS Total_order_value,
ROUND(AVG(orders.payment_value), 2) AS Avg_order_value
FROM orders
LEFT JOIN rank_1_customer
ON orders.customer_id = rank_1_customer.customer_id
GROUP BY rank_1_customer.geolocation_state
ORDER BY total_order_value DESC;
```

Results

Row	geolocation_state	Total_order_value	Avg_order_value
1	SP	5992178.68	137.48
2	RJ	2140500.12	158.63
3	MG	1871072.52	154.75
4	RS	890584.82	157.26
5	PR	810280.15	154.34
6	SC	623086.43	165.98
7	BA	615203.62	170.84
8	GO	348577.6	166.15
9	DF	326895.34	160.87
10	ES	325490.67	154.92

Inference:

Top order value states are SP,RJ,MG,RS,PR. It is also seen that average order value in SP is on lower side even though it is making highest order.

Recommendations:

Need to work on average ticket size as it is on a lower side. Cross selling should be done. Free delivery over a certain order value should be provided to increase average order value. Bundle offers or Family combos can be created.

Q4 (3) Calculate the Total & Average value of order freight for each state.

Query

```
WITH location AS (
SELECT DISTINCT geolocation_zip_code_prefix,geolocation_state
FROM Business_Case_Target_SQL.geolocation
),

customer AS (
SELECT customer_id,customer_zip_code_prefix
FROM Business_Case_Target_SQL.customers
),

orders_with_freight AS (
SELECT o.order_id, o.customer_id, oi.freight_value
FROM Business_Case_Target_SQL.orders o
JOIN Business_Case_Target_SQL.order_items oi
ON o.order_id = oi.order_id
),
```

```
customer_rank AS (
SELECT c.customer_id,l.geolocation_state,
ROW_NUMBER() OVER (PARTITION BY c.customer_id ORDER BY l.geolocation_state) AS rn
FROM customer c JOIN location l
ON c.customer_zip_code_prefix = l.geolocation_zip_code_prefix
),

rank_1_customer AS (SELECT customer_id, geolocation_state
FROM customer_rank
WHERE rn = 1
)

SELECT rank_1_customer.geolocation_state,
ROUND(SUM(orders_with_freight.freight_value), 2) AS total_freight_cost,
ROUND(AVG(orders_with_freight.freight_value), 2) AS avg_freight_cost
FROM orders_with_freight
LEFT JOIN rank_1_customer
ON orders_with_freight.customer_id = rank_1_customer.customer_id
GROUP BY rank_1_customer.geolocation_state
ORDER BY total_freight_cost DESC;
```

Result

Row	geolocation_state	total_freight_cost	avg_freight_cost
1	SP	717871.36	15.14
2	RJ	304744.02	20.96
3	MG	270632.71	20.63
4	RS	135457.56	21.74
5	PR	117660.38	20.54
6	BA	99773.56	26.35
7	SC	89660.26	21.47
8	PE	59307.21	32.91
9	GO	52728.73	22.75
10	ES	49660.73	22.07

Inference:

Top freight value states are SP,RJ,MG,RS,PR. It is also seen that average freight cost in SP is on lower side even though it has highest total freight cost.

Recommendations:

Freight Cost should be reduced by making proper price negotiations with the delivery vendors as it is eating up almost 12% to 14% of Total order value which is reducing profit margins. A small negotiation of almost 2% to 4% will create a huge impact on overall business.

Q5 (1) Find the no. of days taken to deliver each order from the order's purchase date as delivery time.
Also, calculate the difference (in days) between the estimated & actual delivery date of an order. Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

- **time_to_deliver = order_delivered_customer_date - order_purchase_timestamp**
- **diff_estimated_delivery = order_delivered_customer_date - order_estimated_delivery_date**

Query

```
SELECT order_id,
DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY) AS time_to_deliver,
DATE_DIFF(order_delivered_customer_date, order_estimated_delivery_date, DAY) AS
diff_estimated_delivery
FROM Business_Case_Target_SQL.orders;
```

Result

Row	order_id	Delivery_Time	Diff_Delivery_Exp_Delivery
1	65d1e226dfaeb8cdc42f665422...	35	-16
2	2c45c33d2f9cb8ff8b1c86cc28...	30	-28
3	1950d777989f6a877539f53795...	30	12
4	bfbd0f9bdef84302105ad712db...	54	36
5	98974b076b01553d49ee64679...	43	-6
6	c4b41c36dd589e901f6879f25a...	36	-14
7	d2292ff2201e74c5db154d1b7a...	29	-20
8	95e01270fcbae986342340010...	30	-19
9	ed8c7b1b3eb256c70ce0c7423...	44	-5
10	5cc475c7c03290048eb2e742c...	68	18

Insights:

Overall order deliver pace is smooth and lot of orders have been delivered well before the expected delivery date.

Recommendations:

Already eCommerce platforms provide a broad expected delivery window to accommodate potential delays, there are instances where late delivery has occurred. The supply chain and delivery partner's process should be improved.

Q5 (2) Find out the top 5 states with the highest & lowest average freight value.

<u>Query</u>

```
WITH location AS (
  SELECT DISTINCT geolocation_zip_code_prefix, geolocation_state
  FROM Business_Case_Target_SQL.geolocation
),

customer AS (
  SELECT customer_id, customer_zip_code_prefix
  FROM Business_Case_Target_SQL.customers
),

orders_with_freight AS (
  SELECT o.order_id, o.customer_id, oi.freight_value
  FROM Business_Case_Target_SQL.orders o
  JOIN Business_Case_Target_SQL.order_items oi
  ON o.order_id = oi.order_id
),

customer_rank AS (
 SELECT c.customer_id, l.geolocation_state,
 ROW_NUMBER() OVER (PARTITION BY c.customer_id ORDER BY l.geolocation_state) AS rn
 FROM customer c JOIN location l
 ON c.customer_zip_code_prefix = l.geolocation_zip_code_prefix
),

rank_1_customer AS (
  SELECT customer_id, geolocation_state
  FROM customer_rank
  WHERE rn = 1
),

state_freight AS (
  SELECT r.geolocation_state,
  ROUND(AVG(f.freight_value), 2) AS avg_freight_cost
  FROM orders_with_freight f
  JOIN rank_1_customer r ON f.customer_id = r.customer_id
  GROUP BY r.geolocation_state
)

SELECT * FROM (
SELECT * FROM state_freight
ORDER BY avg_freight_cost ASC
 LIMIT 5
)
UNION ALL
SELECT * FROM (
SELECT * FROM state_freight
ORDER BY avg_freight_cost DESC
 LIMIT 5);
```

Result

Row	geolocation_state	avg_freight_cost
1	SP	15.14
2	PR	20.54
3	MG	20.63
4	RJ	20.96
5	DF	21.08
6	RR	42.98
7	PB	42.7
8	RO	41.11
9	PI	39.27
10	MA	38.31

Insights:

The low average freight cost states are SP,PR,MG,RJ & DF. The high average freight cost states are RR,PB,RO,PI & MA. We can also see that the difference of range variation is very high between high avg freight cost states and low average freight cost states.

Recommendations:

Supply chain process should be monitored and see where freight cost can be reduced. Negotiation with delivery partners should also be done. Mode of delivery should be streamlined.

Q5 (3) Find out the top 5 states with the highest & lowest average delivery time.

Query

```
WITH location AS (
  SELECT DISTINCT geolocation_zip_code_prefix, geolocation_state
  FROM Business_Case_Target_SQL.geolocation
),

customer AS (
  SELECT customer_id, customer_zip_code_prefix
  FROM Business_Case_Target_SQL.customers
),

orders AS (
  SELECT order_id, customer_id,
  DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY) AS delivery_time
  FROM Business_Case_Target_SQL.orders
),
```

```
customer_rank AS (
  SELECT c.customer_id, l.geolocation_state,
  ROW_NUMBER() OVER (PARTITION BY c.customer_id ORDER BY l.geolocation_state) AS rn
  FROM customer c
  JOIN location l ON c.customer_zip_code_prefix = l.geolocation_zip_code_prefix
),

rank_1_customer AS (
  SELECT customer_id, geolocation_state
  FROM customer_rank
  WHERE rn = 1
),

state_delivery_time AS (
  SELECT r.geolocation_state,
  ROUND(AVG(o.delivery_time), 2) AS avg_delivery_days
  FROM orders o JOIN rank_1_customer r ON o.customer_id = r.customer_id
  GROUP BY r.geolocation_state
)

SELECT * FROM (
 SELECT * FROM state_delivery_time
 ORDER BY avg_delivery_days ASC
 LIMIT 5
)
UNION ALL
SELECT * FROM (
 SELECT * FROM state_delivery_time
 ORDER BY avg_delivery_days DESC
 LIMIT 5
);
```

Result

Row	geolocation_state	avg_delivery_days
1	RR	28.98
2	AP	26.73
3	AM	25.99
4	AL	24.07
5	PA	23.33
6	SP	8.3
7	PR	11.51
8	MG	11.53
9	DF	12.5
10	SC	14.48

Insights:

Fast delivery states are SP,PR,MG,DF & SC. Slow delivery states are RR,AP,AM,AL &PA. Also difference between fast delivering and slow delivery states is huge. It may be due to various factors like geographical condition,delivery infra etc.

Recommendations:

Slow delivery states should work on reducing delivery time by using delivery partners who has fast delivery record in their geography. Also more delivery bases can be made in their geography. Fast delivery state should continue same work and if possible can reduce more delivery time.

Q5 (4) Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.
You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

Query:

```
WITH location AS (
  SELECT DISTINCT geolocation_zip_code_prefix, geolocation_state
  FROM Business_Case_Target_SQL.geolocation
),

customer AS (
  SELECT customer_id, customer_zip_code_prefix
  FROM Business_Case_Target_SQL.customers
),

orders AS (
  SELECT order_id, customer_id, order_delivered_customer_date, order_estimated_delivery_date
  FROM Business_Case_Target_SQL.orders
  WHERE order_delivered_customer_date IS NOT NULL
    AND order_estimated_delivery_date IS NOT NULL
),

customer_ranked AS (
  SELECT
    c.customer_id, l.geolocation_state,
    ROW_NUMBER() OVER (PARTITION BY c.customer_id ORDER BY l.geolocation_state) AS rn
  FROM customer c
  JOIN location l
    ON c.customer_zip_code_prefix = l.geolocation_zip_code_prefix
),

rank_1_customer AS (
  SELECT customer_id, geolocation_state
  FROM customer_ranked
  WHERE rn = 1
)
```

```
SELECT rc.geolocation_state,
  ROUND(AVG(DATE_DIFF(o.order_delivered_customer_date, o.order_estimated_delivery_date,
DAY)), 2) AS avg_delivery_gap
FROM orders o
JOIN rank_1_customer rc
ON o.customer_id = rc.customer_id
GROUP BY rc.geolocation_state
ORDER BY avg_delivery_gap ASC
LIMIT 5;
```

Result

Row	geolocation_state	avg_delivery_gap
1	RO	-19.11
2	AP	-18.73
3	AM	-18.61
4	AC	-17.36
5	RR	-16.41

Insights:

RO,AP,AM,AC & RR are doing super fast deliveries. Delivey vendors and supply chain in these states are doing tremendous job.

Recommendations:

Follow the delivery system followed by these states in slow delivering states.

Q6 (1) Find the month on month no. of orders placed using different payment types.

Query

```
WITH payments AS (
SELECT
p.payment_type,
EXTRACT(YEAR FROM o.order_purchase_timestamp) AS year,
EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month_number,
FORMAT_TIMESTAMP('%B', o.order_purchase_timestamp) AS month_name,
COUNT(o.order_id) AS total_orders
FROM Business_Case_Target_SQL.orders o
JOIN Business_Case_Target_SQL.payments p
ON o.order_id = p.order_id
GROUP BY
p.payment_type,
year,
month_number,
month_name
)
```

```
SELECT
payment_type,
year,
month_name,
total_orders
FROM payments
ORDER BY year, month_number;
```

Results

Row	payment_type	year	month_name	total_orders
1	credit_card	2016	September	3
2	voucher	2016	October	23
3	credit_card	2016	October	254
4	UPI	2016	October	63
5	debit_card	2016	October	2
6	credit_card	2016	December	1
7	credit_card	2017	January	583
8	UPI	2017	January	197
9	debit_card	2017	January	9
10	voucher	2017	January	61

Insights:

Payment by credit cards is 74% followed by UPI 19%. Debit cards and vouchers are less utilised for making payments.

Recommendations:

Debit Card, UPI & voucher based should be promoted by providing good offers or cashback on them.

Q6(2) Find the no. of orders placed on the basis of the payment installments that have been paid.

Query

```
SELECT
payment_installments,
COUNT(Distinct order_id) AS Num_of_orders
FROM Business_Case_Target_SQL.payments
WHERE payment_installments > 0
GROUP BY payment_installments
ORDER BY payment_installments;
```

Result

Row	payment_installments	Num_of_orders
1	1	49060
2	2	12389
3	3	10443
4	4	7088
5	5	5234
6	6	3916
7	7	1623
8	8	4253
9	9	644
10	10	5315

Insights:

More than 45% customers are making one time payments. Installment plan is taken by very less customers. 2 instalment & 3 instalment plan are also have good number of orders.

Recommendations:

Instalment plan selling should be increased and promoted because often big purchases are made in instalments which will help to increase order value. More finance partners should be tied up who can give lucrative deal to customers so they opt for instalment plans.